

Introduction to Prolog Programming: List Processing, Family Relations, and Sorting Algorithms

This document explains the provided Prolog exercises (Exo1.pl, Exo2.pl, and Exo3-4-5.pl). Prolog (Programming in Logic) is a declarative language, meaning you state what the problem is (facts and rules), and the system determines how to solve it.

Core Concepts in Prolog

- **Facts:** Simple truths (e.g., `homme(ali).` - Ali is a man).
- **Rules:** Definitions based on facts and other rules (e.g., `parent(X,Y):- mere(X,Y).` - X is a parent of Y IF X is the mother of Y).
- **Lists:** Ordered collections, represented using the structure `[Head | Tail]`. Head is the first element. Tail is the rest of the list (which is itself a list).
 - Example: The list `[1, 2, 3]` can be seen as `[1 | [2, 3]]`.

Part 1: List Processing (Exo1.pl)

These predicates define common operations on lists, primarily using recursion (a predicate calling itself) to process the list element by element.

1. Basic Utilities

Predicate	Purpose	Logic
<code>cls</code>	Clears the screen.	Executes a terminal command.
<code>appartient(X, L)</code>	Checks if element X is in list L (Membership).	X is the head, OR X belongs to the tail.
<code>premier(X, L)</code>	Finds the first element X.	X is the head of the list <code>[X _]</code> .
<code>dernier(X, L)</code>	Finds the last element X.	Base Case: The list is just <code>[X]</code> . Recursive Case: Ignore the head and find the last element of the tail.
<code>avantdernier(X, L)</code>	Finds the second-to-last element X.	Base Case: The list is <code>[X, _]</code> (exactly two elements). The <code>!</code> (cut) stops further search.
<code>longeur(L, K)</code>	Calculates the length K of list L.	Base Case: Length of <code>[]</code> is 0. Recursive Case: Length of <code>[_ Tail]</code> is <code>1 + length(Tail)</code> .
<code>pair(L)</code>	Checks if list L has an even length.	Recursively consumes two elements at a time.

2. Manipulation and Calculation

Predicate	Purpose	Logic
-----------	---------	-------

Predicate	Purpose	Logic
<code>suppK(K, L1, L2)</code>	Removes the element at position K in L1 to get L2.	K=1: L2 is the tail of L1. K>1: Keep the head, decrement K, and recurse on the tails.
<code>substitue(X, Y, L1, L2)</code>	Replaces all X with Y in L1 to get L2.	If the head of L1 is X, the head of L2 is Y. Otherwise, the head remains the same. Recurse on the tails.
<code>concat(L1, L2, L3)</code>	Concatenates L1 and L2 to form L3.	Takes elements one by one from L1 and adds them to L3 until L1 is empty; then L3 becomes L2.
<code>somme(L, S)</code>	Calculates the sum S of the elements in L.	Base Case: Sum of <code>[X]</code> is X. Recursive Case: <code>S = Head + somme(Tail)</code> .
<code>aumoins2occ(X, L)</code>	Checks if X appears at least twice in L.	Finds the first occurrence of X as the head, then checks if X belongs to the remaining tail using <code>appartient/2</code> .

3. Display and Advanced Logic

Predicate	Purpose	Logic
<code>affiche1(L)</code>	Prints the list elements in forward order.	Prints the head, then recursively calls itself on the tail.
<code>affiche2(L)</code>	Prints the list elements in reverse order.	Calls itself recursively on the tail before printing the head. The output happens as the recursion unwinds.
<code>palindrome(L)</code>	Checks if L is a palindrome (reads the same forward and backward).	Base Cases: <code>[]</code> or <code>[_]</code> are palindromes. Recursive Case: Checks if the first element equals the last element, then removes the last element (<code>dernier, longueur, suppK</code>) and recurses on the smaller list.



Part 2: Family Relations (Exo2.pl)

This exercise uses Prolog's power for defining logical relationships (a knowledge base).

1. Facts (The Knowledge Base)

These predicates establish the basic entities and direct relationships in the family.

- `homme(Name)`: Declares a male person (e.g., `homme(ali)`).
- `femme(Name)`: Declares a female person (e.g., `femme(djamila)`).
- `pere(Father, Child)`: Declares a direct father-child relationship.
- `mere(Mother, Child)`: Declares a direct mother-child relationship.

2. Rules (Derived Relations)

These rules define complex relationships based on the facts.

Predicate	Relation	Logic
parent(X, Y)	X is a parent of Y.	X is the mother of Y OR X is the father of Y.
enfant(X, Y)	X is a child of Y.	Y is a parent of X.
fils(X, Y)	X is a son of Y.	X is male (homme(X)) AND Y is a parent of X.
fille(X, Y)	X is a daughter of Y.	X is female (femme(X)) AND Y is a parent of X.
grand_parent(X, Y)	X is a grandparent of Y.	X is a parent of someone (Z) AND Z is a parent of Y. (Two generations apart).
grand_pere(X, Y)	X is a grandfather of Y.	X is a father (pere(X, Z)) AND Z is a parent of Y.
grand_mere(X, Y)	X is a grandmother of Y.	X is a mother (mere(X, Z)) AND Z is a parent of Y.
frere_ou_soeur(X, Y)	X is a sibling of Y.	X is a brother of Y OR X is a sister of Y.
frere(X, Y)	X is a brother of Y.	X is male, AND X and Y share the same father (Z) AND the same mother (T).
soeur(X, Y)	X is a sister of Y.	X is female, AND X and Y share the same father (Z) AND the same mother (T).
tante(X, Y)	X is an aunt of Y.	X is a sister of Z AND Z is a parent of Y. (An aunt is the sister of one of the parents).
cousin_cousine(X, Y)	X is a cousin of Y.	Z is a sibling of T AND Z is a parent of X AND T is a parent of Y. (The parents of X and Y are siblings).



Part 3: Sorting and Merging (Exo3-4-5.pl)

These predicates implement sorting algorithms, which are fundamental in computer science, using Prolog's list manipulation capabilities.

1. Merging (For Merge Sort)

Predicate	Purpose	Logic
-----------	---------	-------

Predicate	Purpose	Logic
<code>fusion(L1, L2, L3)</code>	Merges two sorted lists (L1, L2) into a single sorted list (L3).	Base Case: If L1 or L2 is empty, L3 is the non-empty list. Recursive Case: If the head of L1 is less than the head of L2, take the head of L1 and recurse. Otherwise, take the head of L2 and recurse.

2. De-duplication

Predicate	Purpose	Logic
<code>supptoutes(X, L1, L2)</code>	Removes all occurrences of element X from L1 to get L2.	If the head is X, discard it and continue (!). If the head is not X, keep it and continue.
<code>suppocct(L1, L2)</code>	Removes duplicates from a list (L1) to get a list with unique elements (L2).	Takes the head (X), removes all other occurrences of X in the tail, and then recursively calls itself on the rest of the list.

3. Selection Sort

Selection Sort works by repeatedly finding the minimum element from the unsorted part and putting it at the beginning.

Predicate	Purpose	Logic
<code>min(L, Z)</code>	Finds the smallest element Z in list L.	Base Case: If L has one element, that is the minimum. Recursive Case: Compare head with minimum of tail.
<code>suppocct1(X, L1, L2)</code>	Removes only the first occurrence of element X from L1.	If the head is X, remove it (!). If not, keep it and recurse.
<code>triSelection(L1, L2)</code>	Sorts L1 into L2 using Selection Sort.	Find the minimum element (Z) in the input list. Remove the first occurrence of Z from the input list (L3). The sorted list starts with Z, followed by the result of recursively sorting L3.

4. Insertion Sort

Insertion Sort works by building the final sorted array one item at a time. It iterates over the input elements and inserts each element into its correct position in the already sorted list.

Predicate	Purpose	Logic
<code>inserer(X, L_sorted, L_new_sorted)</code>	Inserts element X into an already sorted list (L_sorted) while keeping it sorted.	Base Case: Insert X into <code>[]</code> results in <code>[X]</code> . Recursive Case 1: If $X \leq Y$, place X before Y. Recursive Case 2: Otherwise, keep Y, and recursively find the correct position for X in the rest of the list.
<code>triInsertion(L1, L2)</code>	Sorts L1 into L2 using Insertion Sort.	Recursively sorts the tail of the list (L3), then inserts the head (X) into the sorted tail (L3) using <code>inserer/3</code> .

